Programming SmartNICs: From Packet Processing to Programmable Transport

ACM SIGCOMM 2026

Fernando Ramos, Muhammad Shahbaz, Mina Tahmasbi Arashloo, Mario Baldi

Francisco Pereira, Bilal Saleem, Tyler Liu, Chenxingyu Zhao

INESC-ID, Instituto Superior Técnico, University of Lisbon · University of Michigan · University of Waterloo · NVIDIA · Purdue University · University of Washington

Programmable Congestion Control on the BlueField-3 with DOCA PCC

In Part I you programmed the NIC to *mark* congestion — you set the **CE** bit on packets in the data plane. In this part you write the other side of that story: the **congestion-control algorithm that reacts to those marks**, deciding how fast a flow is allowed to send — and you run it on a set of tiny processors *inside* the NIC called the **DPA**.

By the end you will have **written the two reactions at the heart of a reaction-point controller**, watched a flow's send rate **collapse** the moment congestion appears and **recover** when it clears, and tuned how aggressively it reacts.

We build up in three parts:

- **Part A** — *where* the algorithm runs: the two halves (a host program that loads it, and the DPA that runs it), and how a "rate" is expressed.
- **Part B** — *how* the controller thinks: the event loop and the two reactions that make up a DCQCN-style loop — both of which you'll write.
- **Part C** — fill in the two reactions one at a time: **TODO 1** (the cut) and watch the rate collapse, then **TODO 2** (the recovery) and watch the sawtooth; then tune how hard it reacts.

  Prerequisites. - You have done Part I — PCC reacts to the ECN marks you produced there. - You are on the **Arm cores** of a BlueField-3 with this repo checked out and sudo access. - One firmware knob, **`USER_PROGRAMMABLE_CC=1`**, must be live for any PCC program to start — on the tutorial cards this is already set for you. (Details in the FAQs.)

Part A — Where the algorithm runs

Two halves: a host loader and a DPA program

A DOCA PCC program comes in two pieces that run in two different places:

```
      HOST — the Arm cores (control plane)              NIC hardware (data plane)

 ┌─────────────────────────────────────┐       ┌─────────────────────────────────────────┐
 │ doca_pcc_ecn_rp    (host program)   │ loads │  DPA cores run YOUR algorithm           │
 │   · opens the PCC context           │ ────► │   your_algo()   runs once per EVENT     │
 │   · uploads the DPA program         │       │      ▲ events: a packet sent, a CNP, …  │
 │   · prints logs, stays running      │       │      ▼ writes results->rate             │
 └─────────────────────────────────────┘       │   NIC per-flow RATE LIMITER ◄── set here │
                                               └─────────────────────────────────────────┘
```

- **The host program** (`doca_pcc_ecn_rp`) is just a **loader and supervisor**. It uploads your compiled algorithm to the NIC, opens the PCC context, and then sits there keeping it alive and printing logs.

It does **not** run the algorithm itself.

- **The algorithm** runs on the **DPA** — the *Data-Path Accelerator*, a cluster of small, highly parallel processors inside the BlueField-3. That is where the C code you'll look at actually executes.

  INFO — what is the DPA, and why not the Arm cores? The Arm cores are general-purpose Linux CPUs; they are too far from the wire to make a per-flow rate decision fast enough. The DPA sits right on the data path and is built for exactly this kind of tiny, frequent, reactive computation. You write the algorithm in C, a special compiler (`dpacc`) turns it into a DPA program, and the host loader ships it onto the NIC.

The one thing your algorithm actually outputs: a rate

Your algorithm's whole job is to set **one number per flow**: a target send **rate**. It never touches a packet and never paces anything itself. It writes the rate into a `results` struct, and when it returns, the NIC programs that flow's **hardware rate limiter** to that value. From then on the *hardware* paces the packets — at line speed, with your algorithm nowhere in the loop — until the next event, when your code runs again to revise the number.

> **INFO — rate is a fraction, not a bits-per-second.** The rate is a fixed-point number where **`1 << 20` (= 1,048,576) means "100% of line rate."** So `524288` is 50%, and a small floor value is a near-stop. You will see constants like this in the code; just read `1 << 20` as "full speed."

This split is the key idea of PCC: **you write the *policy* (what the rate should be), the NIC hardware does the *work* (pacing every packet).**

Build it and run it once

You now know the two halves and what the algorithm outputs. Before we open the algorithm itself, **build the whole thing and run it once** — to prove your toolchain works and to *see* the two halves in action: the host loader compiling your DPA program, uploading it to the NIC, and running it live.

> **NOTE — the algorithm is deliberately incomplete right now.** The controller you're about to run is a **scaffold**: the two reactions that move the rate are left blank (you write them in Part C). So it loads, runs, and *sees* congestion — but it won't change the rate yet. That's expected; this step is about the plumbing, not the policy.

Try it yourself! — build the controller and watch it run

Step 1 — build. On the card's Arm cores, from the repo root:

```
cd doca-2 && meson setup build && ninja -C build
# => build/doca-pcc-ecn/doca_pcc_ecn_rp   (the host loader; the DPA program is compiled into
↪  it)
```

That one step compiles the C you'll edit in Part C into a **DPA image** and links it into the host loader. It builds fine even with the two reactions still blank.

Step 2 — load the controller on PF1, in the **foreground**, so all its output prints right here on the terminal:

```
sudo stdbuf -oL ./build/doca-pcc-ecn/doca_pcc_ecn_rp -d mlx5_1 -l 50
```

> **INFO — the flags.** `-d mlx5_1` picks PF1 (the sender's uplink — the "reaction point"); `-l 50` is a chatty log level; `stdbuf -oL` keeps the output line-buffered so it appears as it happens. It runs in the **foreground** — you watch it live and stop it with **Ctrl-C** (that sends SIGINT, the graceful stop; never `kill -9` it — see Debugging Tips).

It opens the PCC context, uploads your DPA program, and then waits for events. With no traffic yet it just sits there — that's expected; a controller with nothing to react to has nothing to do.

Step 3 — give it something to see. In **other terminals** (leave the controller running), start the Part I marker (so packets are CE-marked and the receiver sends CNPs), then drive traffic with `benchmark.sh` — the loopback is already up from Part I:

```
# in a doca-2 terminal — the Part I marker: CE-mark every packet
sudo ./build/doca-flow/doca_flow_ecn_pcap -- --percent 100
# in another terminal (from the repo root) — runs server + client together, with a live
↪  throughput chart
./scripts/benchmark.sh
```

What you should see. Back in the controller's terminal: now that traffic is flowing, its own `PURE_ECN` trace scrolls by — proof your algorithm is running on the DPA and seeing the CNPs:

```
PURE_ECN cnp=1    rate=1048576
PURE_ECN cnp=501  rate=1048576
PURE_ECN cnp=1001 rate=1048576     # it SEES the CNPs, but the rate never moves — no reactions yet
```

Those lines are mixed in with the loader's chatter; if it's too busy to read, stop it and reload with `| grep --line-buffered PURE_ECN` appended to show only these. That flat rate is exactly the point of the checkpoint: **your code compiled, loaded onto the NIC's DPA, and is running** — it just doesn't *react* yet. Making it react is Part C. (`1048576` is `1 << 20` — full line rate, from the INFO box above.)

Step 4 — stop it. In the controller's terminal press **Ctrl-C** (SIGINT — the graceful stop), then stop the marker and traffic in the other terminals.

Part B — How the controller reacts: a DCQCN loop

Start at `main()`: how the controller gets loaded

You ran this program in Part A; here is what it actually does, top to bottom. This is the **host half** — the loader and supervisor — and you won't edit it, but following its flow shows exactly where *your* code (the DPA half) gets plugged in and takes over. It all lives in `host/pcc_ecn_rp.c`:

1. **Read the two flags** — `-d <device>` (which NIC, e.g. `mlx5_1`) and `-l <level>` (log verbosity).
2. **Open that device** — `open_pcc_device()` finds the IB device with that name that *supports PCC*, and opens it.
3. **Create a PCC context** on the device — `doca_pcc_create()`.
4. **Attach your algorithm** — `doca_pcc_set_app(pcc, pcc_ecn_rp_app)`. That `pcc_ecn_rp_app` is the **compiled DPA image**, built from `device/rp_main.c` + your `device/algo/rtt_template.c` — this single line is where the code you write gets loaded in.
5. **Configure it** — the DPA thread pool, the CNP probe format (plain RoCE CNP), logging/coredump.
6. **Start it** — `doca_pcc_start(pcc)` uploads your algorithm onto the NIC's DPA and sets it running. **From this moment the DPA is in charge:** every congestion event runs your code.
7. **Supervise** — the host then just loops in `doca_pcc_wait()`, checking the process stays healthy and otherwise doing nothing. All the real per-event work is on the DPA now.

Steps 1–5 are setup, **step 6 is the handoff**, and step 7 is the host getting out of the way. Everything from here on — the part you actually write — runs on the DPA, once per event. That is what the rest of Part B is about.

It runs once per event, per flow

The DPA calls your algorithm **once per congestion-relevant event, for each flow** — not once per packet. The entry point is `doca_pcc_dev_user_algo()`; it hands your code that flow's saved state, the event, and a `results` struct — you set the new rate by writing it into `results` (the function itself returns nothing). The events that matter here are:

- a **packet was sent** (a "TX" event),
- a **CNP arrived** — a Congestion Notification Packet, the receiver's way of saying "I got a CE-marked packet, you are causing congestion" (this is the mark you set in Part I, echoed back).

  INFO — the NIC batches events for you. At line rate the DPA could never keep up with one event per packet, so the hardware **coalesces** them: one event stands for many packets. That is what keeps the algorithm fast enough — it reacts per *batch*, while the hardware rate limiter does the per-packet work.

How an event reaches your handler

You don't wire any of this up. The DPA calls **one fixed entry point** per event, and a small dispatch routes it to the right handler by **event type**. Here's the path a **CNP** takes to the code you'll write (`TODO 1`), and where the other events land:

```
a CNP arrives at the NIC   (the receiver echoing back your Part I CE mark)
              │
              ▼  NIC coalesces events, then calls your entry point — once, for this flow:
doca_pcc_dev_user_algo(algo_ctxt, event, attr, results)                      ← rp_main.c
     │      algo_ctxt = this flow's saved state       results = where you write the new rate
     ▼
rtt_template_algo(event, …)   reads the event's ev_type, then routes on it     ← rtt_template.c
     │
     ├─ ev_type == ROCE_TX    →  rtt_template_handle_roce_tx()    ← TODO 2 (raise the rate)
     ├─ ev_type == ROCE_CNP   →  rtt_template_handle_roce_cnp()   ← TODO 1 (cut the rate)   ★ this one
     └─ new flow / NACK / …   →  handled for you
              │
              ▼  your handler writes results->rate
the NIC programs this flow's hardware rate limiter to that value
```

So each of the two reactions below is just the body of one of those `..._handle_roce_*` functions — the dispatch that gets you there is already written.

The two reactions

Our controller is a textbook **DCQCN** loop — the classic "additive-increase / multiplicative- decrease" pattern. All of it lives in two short handlers in `rtt_template.c`, and each does one thing. **Both of them are left blank for you** — that's the exercise — so below we describe exactly what each must do and the pieces you build it from. You'll write them in Part C; here, just take in the shape.

1. A CNP arrives → cut the rate (multiplicative decrease). ← you write this, `TODO 1`. Congestion is happening, so the rate must come **down** by a fixed factor, and never fall below a floor. It goes in `..._handle_roce_cnp()`, at the marker `TODO 1`. The pieces, all already there for you: - `cur_rate` — the flow's current rate (the fixed-point number from Part A); you edit it in place. - `ECN_CNP_DEC_FACTOR` — the cut factor, ×0.90 by default (a `#define` at the top of the file). - `doca_pcc_dev_fxp_mult(factor, rate)` — multiplies that fixed-point `factor` into a rate for you. - `MIN_RATE` — the floor the rate must not drop below.

2. Packets keep flowing, no congestion → raise the rate (additive increase). ← you write this, `TODO 2`. When traffic is moving and nothing is cutting it, the rate should **drift back up** — gently, and only occasionally, so it doesn't overshoot and immediately re-trigger congestion. It goes in `..._handle_roce_tx()`, at the marker `TODO 2`. The pieces: - a `static` counter, so you act only every ~1000th call rather than on every single send event; - `AI >> 2` — a small step to add each time, about 1.25% of line rate; - `RATE_MAX` — the ceiling the rate must not exceed.

Those two reactions are the entire controller: it steers the rate on the ECN signal alone (the CNPs) — which is what *pure-ECN* means. Put together, they make the DCQCN **sawtooth**: every CNP knocks the rate down by 10%, and between CNPs it drifts back up ~1.25% at a time. When the network is congested, the down-cuts win and the rate settles low; when congestion clears (no more CNPs), the rate climbs back to full.

> **INFO — the one knob that changes its personality.** The decrease factor is a single constant at the top of the file, `ECN_CNP_DEC_FACTOR`:
>
> ```
> #define ECN_CNP_DEC_FACTOR (((1 << 16) * 900) / 1000)  // ×0.90 per CNP; 800..995
> ↪  = ×0.80..×0.995
> ```
>
> `900` means ×0.90. Make it smaller and each CNP cuts harder; larger and it barely reacts. You'll use this constant in **TODO 1**, and tune it at the end of Part C.

Part C — Write the two reactions, then watch it work

The controller is **almost complete**: the event loop, the logging, and the whole host loader are done. **The two reactions that actually move the rate are left for you** — the ones you studied in Part B:

- **`TODO 1`** — cut the rate when a CNP arrives (multiplicative decrease), in `..._handle_roce_cnp()`
- **`TODO 2`** — raise the rate when things are quiet (additive increase), in `..._handle_roce_tx()`

Both are marked in `device/algo/rtt_template.c`. As shipped they're empty, so the controller **builds and runs but never changes the rate** — a flow sends flat-out no matter how congested the link is. You'll add them one at a time and watch each half of the loop come alive.

The pieces you'll have running together while you test:

what	where	why
the ECN marker from Part I (`doca_flow_ecn_pcap`)	PF0 (`mlx5_0`)	marks packets CE, which makes the receiver send CNPs
the PCC controller (`doca_pcc_ecn_rp`)	PF1 (`mlx5_1`)	reacts to those CNPs by setting the sender's rate
RoCE traffic (`ib_write_bw`)	the SFs (`mlx5_2`/`mlx5_3`)	the flow whose rate you'll watch move

Stage 1 — write `TODO 1` *(the cut) and watch the rate collapse*

In the Part A checkpoint you saw the scaffold run and its rate stay **flat** under a flood of CNPs — it sees congestion but never reacts. Now you write the reaction that makes it react.

Try it yourself! — make the controller react to congestion

Step 1 — get the flow running, and note the baseline. Bring the same setup back up as in the Part A checkpoint — the Part I marker at `--percent 100` and `./scripts/benchmark.sh` — and note the baseline throughput on its chart (~92 Gb/s, "full speed"). As you saw there, with `TODO 1` still empty the rate sits flat and the BW doesn't budge. Leave the traffic and marker running while you edit.

Step 2 — write `TODO 1`. Open `device/algo/rtt_template.c` and find `TODO 1` in `..._handle_roce_cnp()`. Using the pieces from Part B — `doca_pcc_dev_fxp_mult()`, `ECN_CNP_DEC_FACTOR`, `cur_rate`, `MIN_RATE` — make the rate come **down** by the cut factor on each CNP, then clamp it up to the floor so it can't go below `MIN_RATE`. It's two lines. (Stuck? The Check your answer block below has it.)

Step 3 — rebuild, reload, and watch it collapse. Rebuild, then load the controller again in the foreground (the traffic and marker from Step 1 are still running):

```
ninja -C build
sudo stdbuf -oL ./build/doca-pcc-ecn/doca_pcc_ecn_rp -d mlx5_1 -l 50
```

NOTE — the DPA image is built at *configure* time. If an edit doesn't seem to take effect, do a clean rebuild: `rm -rf build && meson setup build && ninja -C build`.

Now you should see two things:

1. **The throughput drops sharply.** With the controller cutting the rate on every CNP, the client's BW average falls well below your baseline (it at least halves, usually much more). *That is congestion control happening* — the sender is throttling itself.
2. **The rate walking down in the controller's terminal** — the multiplicative-decrease half of the loop, live (the `PURE_ECN cnp=… rate=…` lines, no longer flat):
```
PURE_ECN cnp=1    rate=943718
PURE_ECN cnp=501  rate=214380
PURE_ECN cnp=1001 rate=52428
...
```

Stop the controller with **Ctrl-C** (SIGINT — the graceful stop) when you're done looking.

Stage 2 — write `TODO 2` (the recovery) and complete the sawtooth

With only `TODO 1`, the rate can only ever fall: once CNPs cut it, it stays down even after congestion clears. `TODO 2` adds the other half — the gentle climb back up — so the controller can *recover*.

Try it yourself! — let the rate come back

Step 1 — write `TODO 2`. Find `TODO 2` in `..._handle_roce_tx()`. Using the pieces from Part B — a `static` counter, `AI >> 2`, `RATE_MAX` — every ~1000th call, add the small step to `cur_rate` and cap it at `RATE_MAX`. Acting only every ~1000th call is what keeps the increase *gentle* — leave the gate out and the rate rockets straight back up and re-triggers congestion.

Step 2 — rebuild: `ninja -C build`.

Step 3 — run, and this time make congestion come and go. Load the controller (foreground) and traffic as in Stage 1, then, partway through, **stop the marker** in its terminal so the CNPs dry up while the flow keeps running:

```
sudo pkill -INT -x doca_flow_ecn_pcap    # congestion clears; no more CNPs
```

Watch the controller's terminal: while the marker ran the rate was cut down (that's `TODO 1`); once the CNPs stop, the rate **climbs back toward full** (that's `TODO 2`). That rise-and-fall is the DCQCN **sawtooth** — both halves of your controller working together:

```
PURE_ECN cnp=1001 rate=52428      # cut down while marking
# ... marker stopped: no new PURE_ECN cuts, and TX events raise the rate back toward 1048576 ...
```

Stop with **Ctrl-C** (SIGINT — the graceful stop).

Check your answer

Show the two finished reactions

`TODO 1` — in `..._handle_roce_cnp()`:

```
cur_rate = doca_pcc_dev_fxp_mult(ECN_CNP_DEC_FACTOR, cur_rate);   // rate = rate × 0.90
if (cur_rate < MIN_RATE) cur_rate = MIN_RATE;                     // never below the floor
```

`TODO 2` — in `..._handle_roce_tx()`:

```
static uint32_t g_tx_inc = 0;
if ((++g_tx_inc % 1000) == 0) {          // every ~1000th send event
```

```
    cur_rate += (AI >> 2);              // add ~1.25% of line rate
    if (cur_rate > RATE_MAX) cur_rate = RATE_MAX;
}
```

Or diff your file against the finished controller — it ships in the **solutions** repository at the same path:

```
diff device/algo/rtt_template.c
↪  <solutions-repo>/doca-2/doca-pcc-ecn/device/algo/rtt_template.c
```

Going further (optional) — tune how hard it cuts

Now that the loop works, change **how aggressively** it reacts. Every CNP currently cuts the rate by 10% (`ECN_CNP_DEC_FACTOR = ×0.90`). This one number sets the controller's whole personality: a sharper cut empties the queue faster (fewer retransmissions, higher goodput, lower latency) up to a point; too sharp and you under-use the link.

Try it yourself! — sweep the cut factor

Open `device/algo/rtt_template.c` and change the `900` in `ECN_CNP_DEC_FACTOR` (near the top of the file). Try a gentler reaction first, rebuild, and re-run Stage 1:

```
#define ECN_CNP_DEC_FACTOR (((1 << 16) * 990) / 1000)  // ×0.99 — barely cuts per CNP
```

With ×0.99 the rate barely comes down, the queue stays deep, and you'll see **far more CNPs** in the controller's `PURE_ECN` trace and lower goodput than at ×0.90. Now try a **sharper** cut (`800 = ×0.80`) and compare — fewer CNPs, a shallower queue. The tuned sweet spot on our testbed was **×0.90**:

per-CNP cut	goodput	CNPs	latency
×0.99	68 Gb/s	many	high
×0.90	**88.5 Gb/s**	few	**low**
×0.80	86.7 Gb/s	fewest	very low

INFO — the surprising bit. The rate *set-point* averaged about the same across all of these, yet goodput ranged from 68 to 88 Gb/s. Goodput is governed by **queue depth and retransmissions**, not the average rate — which is why a *sharper* cut (shallower queue, fewer drops) can *raise* goodput. The `tune_ecn.py` script and `doca_pcc_ecn_sweep.pdf` sweep this automatically.

Put `900` back when you're done to restore the tuned controller.

What you built (and saw)

- The **two-halves** model: a host loader ships your algorithm onto the **DPA**, which runs it once per event and sets a per-flow **rate** that the NIC hardware then enforces.
- The two reactions **you wrote** — `TODO 1` (CNP → cut ×0.90) and `TODO 2` (TX → raise ~1.25%) — which together make a complete **DCQCN** sawtooth.
- **Congestion control happening live**: with only the cut written, a flow's throughput collapsing as the controller reacts to the CE marks you produced in Part I; with the recovery added too, the rate climbing back when congestion clears.
- How **one constant** changes the whole behaviour, and why a sharper cut can *improve* goodput.

Debugging Tips

- **The rate never moves — `PURE_ECN` prints the same number on every line** → `TODO 1` is still empty (or you edited it but didn't rebuild). That flat output is the scaffold's as-shipped behaviour; write the multiplicative decrease in `..._handle_roce_cnp()` and rebuild. If instead the rate falls but never climbs back after congestion clears, `TODO 2` (the increase, in `..._handle_roce_tx()`) is the missing half.
- **The controller exits a second or two after starting** → the firmware knob `USER_PROGRAMMABLE_CC` is not live. Check with `admin/local_scripts/check_pcc_ready.sh` (it should say `ready`). On the tutorial cards it's set for you; if you see `doca_pcc … not supported`, that's the cause.
- **No `PURE_ECN` lines ever appear** → no CNPs are reaching the controller. Make sure the Part I marker (`doca_flow_ecn_pcap --percent 100`) is running so packets are CE-marked, and that traffic is actually flowing. (If the controller's output is just too chatty to spot them, reload it with `| grep --line-buffered PURE_ECN` appended to see only that trace.)
- **Stop it gracefully with Ctrl-C** in its terminal (SIGINT) — or `sudo pkill -INT -x doca_pcc_ecn_rp` from another terminal. A hard `kill -9` (or `docker rm -f`) leaves a "ghost" program loaded on the DPA that blocks the next run until a chip reset.
- **Run it in the foreground** (as shown), so its output prints live on the terminal. Launched in the background over SSH it can appear to die after a few seconds — that's the launch, not the program; the foreground keeps it attached and visible.
- **An edit to the algorithm didn't take effect** → the DPA image is built at *configure* time, so do a clean rebuild: `rm -rf build && meson setup build && ninja -C build`.
- **Traffic must use `-R`** (`benchmark.sh` and `run_{server,client}.sh` already do). Without it, the flow isn't bound to your algorithm and your handlers never run — the rate won't move at all.

FAQs

What is `USER_PROGRAMMABLE_CC` and why is it needed? It's a NIC firmware setting that enables running your *own* congestion-control code on the DPA. Without it, `doca_pcc_ecn_rp` refuses to start. It only goes live after a full power-cycle of the card, so it's set up for you ahead of the tutorial.

Why does the controller attach to `mlx5_1` (PF1), but traffic uses `mlx5_2`/`mlx5_3` (the SFs)? Congestion control is a property of the *port/uplink*, so the controller attaches to the sender's physical function (PF1). The traffic rides on sub-functions of that port. One controller governs the flows on its port.

Why does the rate collapse so much at `--percent 100`? Because you're marking *every* packet, the receiver sends a constant stream of CNPs, so the controller thinks the network is maximally congested and keeps cutting. Mark a smaller fraction (`--percent 50`) for a gentler, more realistic signal.

Does my algorithm run for every packet? Is the DPA fast enough? No — the NIC **coalesces** events, so your code runs per *batch*, far below the packet rate, and the hardware rate limiter does the per-packet pacing. That two-tier split is exactly why it keeps up at line rate.

Why is the rate register not the same as the goodput I measure? The rate is a *set-point*; the throughput you get also depends on queueing and retransmissions. A controller can hold a similar average rate yet deliver very different goodput — which is what the Stage 2 sweep shows.

That's the pair: in **Part I** you marked the congestion signal in the data plane, and here in **Part II** you wrote and tuned the controller that reacts to it on the DPA — the two halves of programmable transport on a SmartNIC.